



MODÉLISATION ET PROTOCOLE EXPÉRIMENTAL 1

1 – Étude d'un cas simple mono-agent avec l'algorithme SAC

SUJET DU STAGE : Étude d'algorithmes d'apprentissage par renforcement pour l'optimisation en temps du transport des bagages par des véhicules guidés automatisés (AGV) en environnement aéroportuaire.

AUTEURE : Sayan Suos

Table des matières

Introduction	2
1 Modélisation du problème	3
1.1 Espace des états	3
1.2 Espace des actions	4
1.3 Fonction de récompense	4
2 Apprentissage avec l’algorithme Soft Actor-Critic	6
2.1 Apprentissage de la fonction de valeur d’action Q_ϕ	7
2.2 Apprentissage de la politique π_θ	8
2.3 Architecture des réseaux	8
3 Protocole expérimental	12
3.1 Curriculum d’entraînement	12
3.2 Environnements de test	14
3.3 Critères d’évaluation	14
4 Résultats	15
A Annexes	18
A.1 Paramètres du simulateur	18
A.2 Pseudo-code de l’algorithme SAC	19

Introduction

Le transport de bagages par des véhicules guidés automatisés (AGV) en environnement aéroportuaire soulève des problématiques de navigation multi-agents en environnement dynamique. Au sein de notre projet, l'objectif est de minimiser le temps de transport tout en garantissant l'absence de collision, dans un cadre décentralisé où les agents ne disposent que d'observations locales et ne communiquent pas explicitement entre eux. Dans cette optique, un simulateur est actuellement développé afin de modéliser l'environnement étudié et d'expérimenter différentes approches d'apprentissage par renforcement.

Dans un premier temps, nous nous intéressons à un cadre plus général de navigation autonome, en considérant des robots mobiles autonomes (AMR). Cette approche permet d'étudier et de valider les méthodes d'apprentissages dans un environnement plus libre, avant de les adapter progressivement aux contraintes spécifiques du contexte aéroportuaire.

Dans cette première phase expérimentale, nous choisissons d'étudier un algorithme classique d'apprentissage par renforcement profond afin de prendre en main le simulateur et d'implémenter les principaux éléments du problème. Plus précisément, nous entraînons un agent unique dans un environnement comportant des obstacles statiques et dynamiques à l'aide de l'algorithme Soft Actor-Critic (SAC). Ce choix est motivé par le fait que SAC est une méthode off-policy, ce qui permet une meilleure efficacité d'apprentissage. De plus, elle est particulièrement adaptée aux espaces d'actions continues et est largement utilisée dans la littérature pour ses bonnes performances et sa stabilité (1; 2).

1 Modélisation du problème

Le problème est modélisé comme un processus de décision de Markov (MDP), représenté par le tuple (S, A, P, R, γ) où :

- S est l'ensemble des états qui vont permettre à l'agent de sélectionner une action,
- A est l'ensemble des actions qui correspondent aux décisions de l'agent,
- $P : S \times A \times S \rightarrow [0, 1]$ est la probabilité de transition entre les états, qui modélise la densité de probabilité du prochain état s_{t+1} , conditionnellement à l'état courant s_t et l'action a_t ,
- $R : S \times A \rightarrow [r_{\min}, r_{\max}]$ est la fonction de récompense qui attribue une valeur à chaque transition,
- et γ est le facteur d'actualisation qui modélise l'importance du futur.

L'objectif de l'agent est **d'atteindre la position cible** (x_g, y_g) **tout en évitant toute collision**, en sélectionnant une action à chaque instant t .

1.1 Espace des états

À chaque instant t , l'agent observe un état $s_t \in S$ défini par :

$$s_t = (\mathbf{M}_t, d_t, \mathbf{e}_t, \mathbf{v}_t, \mathbf{o}_t),$$

où :

- $\mathbf{M}_t \in \{0, 1\}^{K \times W \times H}$ représente l'environnement local de l'agent construit à partir des K dernières observations. Chaque observation est une carte encodée de taille $W \times H$, centrée sur l'agent. Une cellule vaut 1 lorsqu'elle est occupée par un obstacle et 0 sinon.
- $d_t \in \mathbb{R}^+$ représente la distance euclidienne à l'objectif :

$$d_t = \sqrt{(x_g - x_t)^2 + (y_g - y_t)^2},$$

où (x_t, y_t) désigne la position de l'agent et (x_g, y_g) la position de la cible.

- $\mathbf{e}_t = (\cos(\Delta\theta_t), \sin(\Delta\theta_t))$ représente l'erreur d'orientation de l'agent par rapport à l'objectif, avec :

$$\Delta\theta_t = \theta_t^g - \theta_t,$$

où θ_t^g désigne la direction de l'objectif dans le repère global.

- $\mathbf{v}_t = (v_t, \omega_t)$ représente respectivement la vitesse linéaire et la vitesse angulaire du robot.
- $\mathbf{o}_t = (\cos(\theta_t), \sin(\theta_t))$ représente l'orientation du robot, où θ_t correspond à son angle d'orientation dans le repère global.

Les angles sont représentés à l'aide de leurs composantes trigonométriques afin d'éviter les discontinuités associées à une représentation angulaire directe. L'orientation du robot θ_t , est définie sur l'intervalle $[-\pi, \pi]$. Les commandes de contrôle sont contraintes par les limites physiques du robot telles que $v_t \in [v_{\min}, v_{\max}]$ et $\omega_t \in [\omega_{\min}, \omega_{\max}]$.

1.2 Espace des actions

À chaque instant t , l'agent choisit une action $\mathbf{a}_t \in A$ qui correspond aux commandes de contrôle appliquées au robot :

$$\mathbf{a}_t = (v_t, \omega_t),$$

où v_t et ω_t désignent respectivement la vitesse linéaire et la vitesse angulaire du robot, avec $v_t \in [v_{\min}, v_{\max}]$ et $\omega_t \in [\omega_{\min}, \omega_{\max}]$.

1.3 Fonction de récompense

La fonction de récompense est définie comme la somme de plusieurs termes :

$$R = R_g + R_r + R_s + R_c,$$

où R_g récompense la progression vers l'objectif, R_r pénalise les rotations importantes, R_s pénalise la proximité des obstacles et R_c pénalise les collisions.

Les notations suivantes sont définies :

- d_t^g désigne la distance entre le robot et la cible à l'instant t ,
- d_t^o désigne la distance entre le robot et l'obstacle le plus proche à l'instant t ,
- ξ représente la distance minimale de sécurité.

Récompense de progression

La récompense de progression est définie par :

$$R_g = \begin{cases} \beta_1 \times (d_{t-1}^g - d_t^g) + 2 \times v_t - 1 & \text{si } d_t^o \geq \xi_s \\ 0.2 \times \beta_1 \times (d_{t-1}^g - d_t^g) + 2 \times v_t - 1 & \text{sinon.} \end{cases}$$

Un bonus supplémentaire est attribué lorsque l'objectif est atteint (i.e. $d_t^g < \xi_g$) :

$$R_g \leftarrow R_g + r_{\text{goal_reward}}, \quad \text{où } r_{\text{goal_reward}} > 0.$$

Cette formulation récompense la diminution de la distance à la cible. Lorsque le robot évolue à proximité d'un obstacle, la récompense de progression est volontairement réduite afin de privilégier la sécurité. Une pénalité constante de -1 est appliquée à chaque étape afin d'encourager l'agent à atteindre sa cible en un nombre minimal d'actions. Enfin, un terme proportionnel à la vitesse linéaire favorise des déplacements rapides lorsque ceux-ci sont compatibles avec les autres contraintes.

Pénalité de rotation

Afin d'éviter les rotations inutiles, une pénalité quadratique est appliquée à la vitesse angulaire :

$$R_r = \beta_2 \times r_{\text{angular_malus}} \times \omega_t^2, \quad \text{où } r_{\text{angular_malus}} < 0.$$

Cette pénalité limite les rotations excessives et favorise des trajectoires plus fluides.

Pénalité de sécurité

Lorsque la distance au plus proche obstacle devient inférieure au seuil de sécurité ξ , une pénalité est appliquée :

$$R_s = \begin{cases} \beta_3 \times r_{\text{safety_malus}} \times \left(\frac{1}{(d_t^o)^2} - \frac{1}{\xi^2} \right) & \text{si } d_t^o < \xi_s, \\ 0 & \text{sinon,} \end{cases} \quad \text{où } r_{\text{safety_malus}} < 0.$$

Cette pénalité augmente rapidement lorsque le robot se rapproche d'un obstacle, ce qui l'incite à conserver une marge de sécurité.

Pénalité de collision

Enfin, une forte pénalité est attribuée en cas de collision :

$$R_c = \begin{cases} r_{\text{collision_malus}} & \text{si } d_t^o < \xi_c, \\ 0 & \text{sinon,} \end{cases} \quad \text{où } r_{\text{collision_malus}} < 0.$$

Cette pénalité vise à décourager fortement toute trajectoire conduisant à une collision.

Les coefficients de cette fonction de récompense ont été déterminés empiriquement au cours des différentes phases d'expérimentation, en s'appuyant sur les principes proposés dans la littérature (3; 4; 5). L'objectif est d'obtenir un compromis entre rapidité de déplacement, fluidité des trajectoires et sécurité vis-à-vis des obstacles.

2 Apprentissage avec l'algorithme Soft Actor-Critic

Une politique $\pi(\mathbf{a}_t | \mathbf{s}_t)$ définit le comportement de l'agent en associant à chaque état une distribution de probabilité sur les actions. En interagissant avec l'environnement, cette politique génère des trajectoires :

$$\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \mathbf{a}_1, \dots).$$

On note $\rho_\pi(\mathbf{s})$ la distribution des états visités lorsque l'agent suit la politique π , et $\rho_\pi(\mathbf{s}, \mathbf{a})$ la distribution conjointe des couples état-action rencontrés au cours de ces trajectoires.

Les algorithmes Actor-Critic reposent sur la combinaison de : (i) un actor network qui prend l'état $\mathbf{s}$ en entrée et génère une action $\mathbf{a}$ en modélisant la politique $\pi_\theta(\mathbf{a} | \mathbf{s})$, et (ii) un critic network qui prend en entrée l'état $\mathbf{s}$ et l'action $\mathbf{a}$ afin d'évaluer cette politique en estimant la fonction de valeur d'action $Q_\phi(\mathbf{s}, \mathbf{a})$.

Dans le cas de SAC, les interactions avec l'environnement génèrent des transitions de la forme :

$$(\mathbf{s}, \mathbf{a}, r, \mathbf{s}', d),$$

où $\mathbf{s}'$ désigne l'état suivant obtenu après l'exécution de l'action $\mathbf{a}$ dans l'état $\mathbf{s}$, r la récompense associée à cette transition, et $d \in \{0, 1\}$ indique si $\mathbf{s}'$ est terminal ou non. Ces transitions sont stockées dans un replay buffer $\mathcal{D}$ afin d'être réutilisées durant l'apprentissage.

En apprentissage par renforcement, l'objectif classique consiste à maximiser la somme des récompenses cumulées espérées. L'algorithme Soft Actor-Critic généralise cet objectif en intégrant un terme d'entropie. Cette régularisation encourage l'exploration en favorisant des politiques plus stochastiques et permet d'améliorer la robustesse de l'apprentissage.

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t (R(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_t))) \right],$$

où l'entropie de la politique dans l'état $\mathbf{s}$ est défini par :

$$\mathcal{H}(\pi) = \mathbb{E}_{\mathbf{a} \sim \pi(\cdot | \mathbf{s})} [-\log \pi(\mathbf{a} | \mathbf{s})].$$

Le paramètre $\alpha > 0$, appelé paramètre de température, contrôle le compromis entre exploitation et exploration. Il peut être soit fixé avant l'apprentissage, soit ajusté durant l'apprentissage afin d'atteindre une entropie cible.

Dans ce cadre, les fonctions de valeur associées à une politique π sont définies par :

$$V^\pi(\mathbf{s}) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(R(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_t)) \right) \middle| \mathbf{s}_0 = \mathbf{s} \right],$$

$$Q^\pi(\mathbf{s}, \mathbf{a}) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t R(\mathbf{s}_t, \mathbf{a}_t) + \alpha \sum_{t=1}^{\infty} \mathcal{H}(\pi(\cdot | \mathbf{s}_t)) \middle| \mathbf{s}_0 = \mathbf{s}, \mathbf{a}_0 = \mathbf{a} \right].$$

Ainsi, les fonctions V^π et Q^π sont liées de la manière suivante :

$$V^\pi(\mathbf{s}) = \mathbb{E}_{\mathbf{a} \sim \pi} [Q^\pi(\mathbf{s}, \mathbf{a}) + \alpha \mathcal{H}(\pi(\mathbf{a} | \mathbf{s}))]$$

et l'équation de Bellman de Q^π est :

$$Q^\pi(\mathbf{s}, \mathbf{a}) = \mathbb{E}_{\mathbf{s}' \sim P, \mathbf{a}' \sim \pi} [R(\mathbf{s}, \mathbf{a}) + \gamma(Q^\pi(\mathbf{s}', \mathbf{a}') + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}')))] = \mathbb{E}_{\mathbf{s}' \sim \rho_\pi} [R(\mathbf{s}, \mathbf{a}) + \gamma V^\pi(\mathbf{s}')]]$$

2.1 Apprentissage de la fonction de valeur d'action Q_ϕ

L'équation de Bellman pour la fonction de valeur d'action peut être réécrite comme :

$$Q^\pi(\mathbf{s}, \mathbf{a}) = \mathbb{E}_{\mathbf{s}' \sim P, \mathbf{a}' \sim \pi} [R(\mathbf{s}, \mathbf{a}) + \gamma(Q^\pi(\mathbf{s}', \mathbf{a}') - \alpha \log \pi(\mathbf{a}' | \mathbf{s}'))]$$

Cela signifie que les actions très probables entraînent un petit bonus alors que les actions peu probables entraînent un plus grand bonus, favorisant l'exploration.

Cette équation permet de définir une cible d'apprentissage. En pratique, l'espérance est approximée par échantillonnage :

$$Q^\pi(\mathbf{s}, \mathbf{a}) \approx r + \gamma(Q^\pi(\mathbf{s}', \tilde{\mathbf{a}}') - \alpha \log \pi(\tilde{\mathbf{a}}' | \mathbf{s}')), \quad \tilde{\mathbf{a}}' \sim \pi(\cdot | \mathbf{s}'),$$

où les transitions $(\mathbf{s}, \mathbf{a}, r, \mathbf{s}', d)$ sont échantillonnées dans le replay buffer $\mathcal{D}$, tandis que l'action suivante $\tilde{\mathbf{a}}'$ est générée par la politique courante.

Afin de limiter le biais de surestimation de la fonction de valeur, SAC utilise le principe du double-Q learning : deux fonctions de valeur paramétrées par ϕ_1 et ϕ_2 sont apprises simultanément, et seule la plus petite estimation est utilisée pour calculer la cible d'apprentissage. La fonction de perte du critique est alors donnée par :

$$L(\phi_i) = \mathbb{E}_{(\mathbf{s}, \mathbf{a}, r, \mathbf{s}', d) \sim \mathcal{D}} \left[(Q_{\phi_i}(\mathbf{s}, \mathbf{a}) - y(r, \mathbf{s}', \mathbf{a}))^2 \right],$$

où la cible est donnée par :

$$y(r, \mathbf{s}', \mathbf{a}) = r + \gamma(1 - d) \left(\min_{i \in \{1, 2\}} Q_{\phi_{i, \text{target}}}(\mathbf{s}', \tilde{\mathbf{a}}') - \alpha \log \pi(\tilde{\mathbf{a}}' | \mathbf{s}') \right), \quad \tilde{\mathbf{a}}' \sim \pi(\cdot | \mathbf{s}').$$

Afin de stabiliser l'apprentissage, chaque critic network est associé à un réseau cible (target network), dont les paramètres sont mis à jour progressivement selon la règle de soft update :

$$\phi'_{i, \text{target}} \leftarrow (1 - \lambda) \phi_{i, \text{target}} + \lambda \phi_i,$$

où $\lambda \ll 1$ est le coefficient de mise à jour.

2.2 Apprentissage de la politique π_θ

La politique est optimisée de manière à maximiser la fonction de valeur tout en favorisant des politiques de forte entropie. Pour cela, on applique le reparameterization trick, qui permet d'exprimer une action comme une transformation déterministe d'un bruit aléatoire. Plus particulièrement, on utilise une politique gaussienne suivie d'une transformation par tangente hyperbolique (squashed gaussian policy), afin de contraindre les actions dans l'intervalle $[-1, 1]$,

$$\tilde{\mathbf{a}}_\theta(\mathbf{s}, \varepsilon) = \tanh(\mu_\theta(\mathbf{s}) + \sigma_\theta(\mathbf{s}) \odot \varepsilon), \quad \varepsilon \sim \mathcal{N}(0, I),$$

avant leur remise à l'échelle vers les bornes des actions. De cette manière, l'espérance peut être réécrite sous la forme :

$$\mathbb{E}_{\mathbf{a} \sim \pi_\theta} [Q^{\pi_\theta}(\mathbf{s}, \mathbf{a}) - \alpha \log \pi_\theta(\mathbf{a} \mid \mathbf{s})] = \mathbb{E}_{\varepsilon \sim \mathcal{N}} [Q^{\pi_\theta}(\mathbf{s}, \tilde{\mathbf{a}}_\theta(\mathbf{s}, \varepsilon)) - \alpha \log \pi_\theta(\tilde{\mathbf{a}}_\theta(\mathbf{s}, \varepsilon) \mid \mathbf{s})].$$

En remplaçant la fonction Q^{π_θ} par son estimation fournie par les critic networks, l'objectif d'optimisation de l'actor devient alors :

$$\max_{\theta} \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \varepsilon \sim \mathcal{N}} \left[\min_{i \in \{1,2\}} Q_{\phi_i}(\mathbf{s}, \tilde{\mathbf{a}}_\theta(\mathbf{s}, \varepsilon)) - \alpha \log \pi_\theta(\tilde{\mathbf{a}}_\theta(\mathbf{s}, \varepsilon) \mid \mathbf{s}) \right].$$

Durant l'apprentissage, une action est échantillonnée selon la politique afin de maintenir l'exploration. En revanche, durant la phase d'évaluation, la politique est rendue déterministe en utilisant directement la moyenne de la loi gaussienne, ce qui conduit à un comportement plus stable et généralement de meilleures performances.

2.3 Architecture des réseaux

Pour implémenter l'algorithme SAC, deux réseaux de neurones sont utilisés : un actor network, qui paramètre la politique π_θ , et deux critic networks, qui estiment la fonction de valeur d'action Q_ϕ . Dans un premier temps, nous nous inspirons de l'architecture proposée par (6), dans laquelle la politique est apprise à partir de cartes locales égocentriques à l'aide de l'algorithme Distributed Proximal Policy Optimization (DPPO).

Architecture de l'actor network

L'actor network paramètre la politique stochastique π_θ . Il prend en entrée l'observation de l'agent, composée de la carte locale, de la distance normalisée à la cible, de l'erreur d'orientation, des vitesses normalisées et de l'orientation du robot.

La carte locale est d'abord traitée par un extracteur de caractéristiques convolutionnel. Celui-ci est composé d'une couche de convolution, suivie d'une fonction ReLU, puis d'un pooling adaptatif permettant d'obtenir une représentation de taille fixe. Cette représentation est ensuite aplatie et concaténée aux autres variables de l'état.

L'ensemble des informations est ensuite projeté dans un espace latent de dimension fixe. Ce vecteur

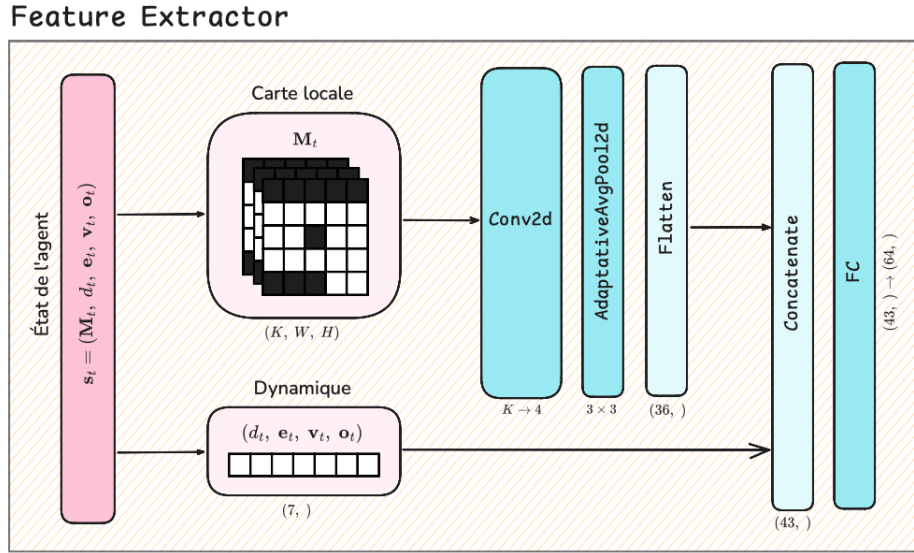


FIGURE 1 – Architecture générale de l’extracteur de caractéristiques convolutionnel

latent est fourni à un réseau entièrement connecté composé de deux couches cachées. La sortie du réseau est divisée en deux branches : une première branche prédit la moyenne $\mu_\theta(\mathbf{s})$ de la distribution d’action, tandis qu’une seconde branche prédit le logarithme de l’écart-type $\log \sigma_\theta(\mathbf{s})$. Le logarithme de l’écart-type est borné afin d’éviter des valeurs trop grandes ou trop faibles, ce qui stabilise l’apprentissage.

À partir de ces deux sorties, une action est échantillonnée selon une loi gaussienne à l’aide du reparametrization trick. L’action est ensuite transformée par une fonction tangente hyperbolique, puis remise à l’échelle afin de respecter les bornes de l’espace d’action. L’action finale correspond donc aux deux commandes du robot : la vitesse linéaire v_t et la vitesse angulaire ω_t .

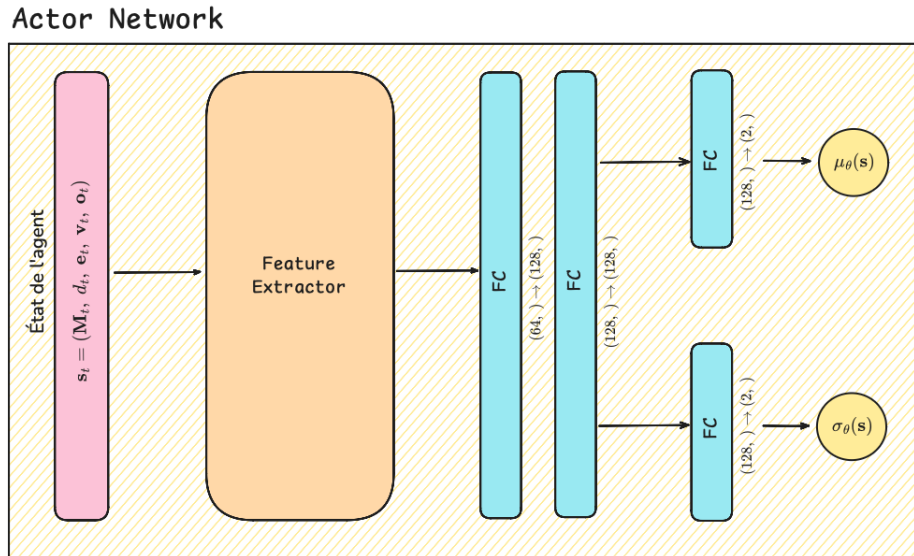


FIGURE 2 – Architecture générale de l’actor network

En phase d’évaluation, l’action n’est plus échantillonnée aléatoirement : on utilise directement la moyenne de la distribution, ce qui rend la politique déterministe et permet d’obtenir un comportement

plus stable.

Architecture des critic networks

SAC utilise deux critic networks indépendants, notés Q_{ϕ_1} et Q_{ϕ_2} . Ces deux réseaux ont la même architecture, mais leurs paramètres sont appris séparément. Leur rôle est d'estimer la valeur d'une action donnée dans un état donné, c'est à dire $Q_{\phi}(s, a)$.

Comme pour l'actor, chaque critic commence par extraire une représentation latente de l'état. La carte locale est traitée par le même type d'extracteur convolutionnel, puis les caractéristiques obtenues sont concaténées avec les autres composantes de l'observation : distance à la cible, erreur d'orientation, vitesses et orientation du robot.

La différence principale avec l'actor est que le critic reçoit également l'action en entrée. Une fois le vecteur latent de l'état obtenu, celui ci est concaténé avec l'action (v_t, ω_t). Le vecteur résultant est ensuite traité par un réseau entièrement connecté composé de deux couches cachées. Cette dernière couche produit une valeur scalaire qui correspond à l'estimation de $Q_{\phi}(s, a)$.

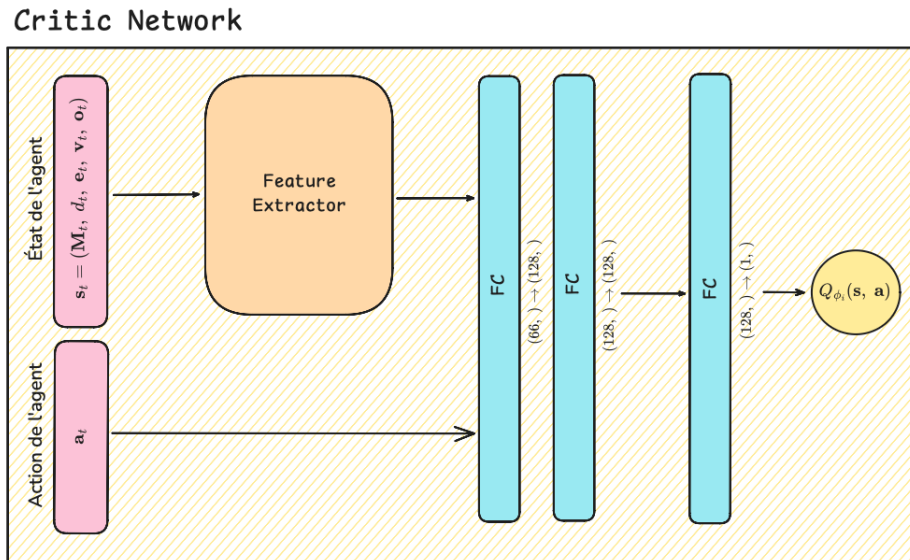


FIGURE 3 – Architecture générale des critic networks

L'utilisation de deux critic networks permet de limiter le biais de surestimation des valeurs Q . Lors du calcul des cibles d'apprentissage et de la mise à jour de l'actor, SAC utilise la plus petite des deux estimations, ce qui rend l'apprentissage plus stable.

Normalisation des observations

Les différentes composantes de l'état présentent des échelles de valeurs hétérogènes. Afin de faciliter l'apprentissage et d'éviter qu'une variable domine les autres, les observations sont normalisées avant d'être fournies aux réseaux de neurones.

- La distance à la cible est normalisée par la diagonale de l'environnement afin d'obtenir une valeur comprise entre 0 et 1.
- Les vitesses linéaire et angulaire sont normalisées par leurs valeurs maximales autorisées.

- Enfin, les variables angulaires (orientation du robot et erreur d'orientation par rapport à la cible) sont représentées par leurs composantes trigonométriques ($\cos(\theta)$, $\sin(\theta)$), ce qui permet d'éviter les discontinuités liées à une représentation angulaire directe.
- Les cartes locales étant binaires, elles ne nécessitent pas de normalisation supplémentaire.

3 Protocole expérimental

Cette section présente le protocole expérimental adopté pour entraîner et évaluer la politique apprise. Dans un premier temps, une stratégie de curriculum learning est mise en œuvre afin d’apprendre progressivement les comportements de navigation. Dans un second temps, la politique est évaluée sur différentes familles d’environnements afin d’analyser ses performances et sa capacité de généralisation

3.1 Curriculum d’entraînement

L’objectif de cette étude est d’entraîner progressivement l’agent à résoudre des tâches de navigation de plus en plus complexes. Pour cela, une stratégie de curriculum learning est adoptée : l’agent est d’abord confronté à des environnements simples, puis la difficulté est augmentée au fur et à mesure de l’apprentissage. Cette approche permet de stabiliser l’entraînement en évitant de confronter immédiatement l’agent à des situations trop complexes. Le curriculum est constitué de sept expériences successives. Les premières expériences visent à apprendre les comportements fondamentaux de navigation, tandis que les dernières introduisent progressivement des environnements plus variés et des interactions plus complexes.

Les scénarios sont caractérisés par la largeur et la hauteur de l’environnement (W_{env} et H_{env}), le nombre d’agents (N_a), d’obstacles statiques (N_s), d’obstacles mobiles (N_m), d’objectifs à atteindre (N_g) ainsi que par le nombre maximal d’étapes par épisode (T_{max}).

TABLE 1 – Récapitulatif des scénarios d’entraînement

<i>Expérience</i>	<i>Objectif</i>	W_{env}	H_{env}	N_a	N_s	N_m	N_g	T_{max}
Exp. 1	Apprendre la navigation vers une cible sans obstacle.	64	32	1	0	0	2	200
Exp. 2	Apprendre un premier contournement avec une colonne de petits obstacles.	64	32	1	3	0	1	110
Exp. 3	Apprendre plusieurs contournements successifs avec deux colonnes d’obstacles et deux objectifs.	64	32	1	6	0	2	200
Exp. 4	Apprendre à naviguer dans des passages plus contraints grâce à des obstacles plus volumineux.	64	32	1	6	0	2	200
Exp. 5	Développer la robustesse face à des obstacles de tailles et de positions variables.	64	32	1	6	0	2	200

<i>Expérience</i>	<i>Objectif</i>	W_{env}	H_{env}	N_a	N_s	N_m	N_g	T_{max}
Exp. 6	Généraliser à des configurations aléatoires composées de quatre obstacles moyens.	64	32	1	4	0	2	200
Exp. 7	Généraliser à des environnements plus denses comportant huit petits obstacles aléatoires.	64	32	8	0	0	2	200

Expérience 1 – Navigation de base : Dans un premier temps, l’agent évolue dans un environnement entièrement libre de tout obstacle. L’objectif est d’apprendre les bases de la navigation, c’est-à-dire rejoindre deux cibles en minimisant le temps de trajet tout en contrôlant correctement sa vitesse linéaire et sa vitesse angulaire. Cette première étape permet également de valider le bon fonctionnement de la fonction de récompense avant d’introduire des contraintes supplémentaires.

Expérience 2 – Premiers comportements d’évitement : Une unique colonne de petits obstacles est ensuite placée au centre de l’environnement. Les positions initiales de l’agent et de la cible sont choisies de manière à ce que la trajectoire directe traverse nécessairement cette colonne. L’agent est ainsi contraint d’apprendre un premier comportement de contournement tout en poursuivant sa progression vers l’objectif.

Expérience 3 – Plusieurs contournements successifs : La difficulté est augmentée en ajoutant une seconde colonne de petits obstacles. Deux objectifs successifs sont introduits : l’agent doit d’abord rejoindre une cible située au milieu de l’environnement, puis une seconde cible située à l’extrémité opposée. Cette configuration impose plusieurs phases de navigation et plusieurs manœuvres d’évitement au sein d’un même épisode.

Expérience 4 – Obstacles de plus grande taille : La disposition générale des obstacles est conservée, mais leur taille est augmentée. Les passages deviennent plus étroits et les marges de manœuvre plus faibles. L’agent doit alors anticiper davantage ses trajectoires et produire des déplacements plus précis.

Expérience 5 – Variabilité des obstacles : Les obstacles conservent une organisation similaire, mais leurs dimensions deviennent variables et leur position est légèrement décalée horizontalement ou verticalement. Cette étape vise à éviter que l’agent ne mémorise une configuration particulière de l’environnement et à améliorer sa robustesse face à de légères variations géométriques.

Expérience 6 – Premières généralisations : Les obstacles ne sont plus prédéfinis mais générés aléatoirement à chaque épisode. Quatre obstacles de taille moyenne sont placés en respectant les contraintes de génération (absence de recouvrement, marge minimale entre les objets, etc.). L’agent doit alors apprendre une stratégie de navigation capable de s’adapter à des configurations variées.

Expérience 7 – Environnement fortement encombré : Le nombre d’obstacles générés aléatoirement est augmenté. Huit petits obstacles sont placés aléatoirement dans l’environnement, créant des scènes plus denses et plus variées. Cette étape vise à développer une politique robuste capable

de naviguer efficacement dans des environnements fortement encombrés.

3.2 Environnements de test

Les performances de la politique apprise sont évaluées sur plusieurs familles d'environnements présentant des niveaux de difficulté et des caractéristiques différentes. L'objectif est d'analyser non seulement les performances de navigation obtenues dans des scénarios similaires à ceux utilisés lors de l'entraînement, mais également la capacité de généralisation de l'agent sur des configurations plus variées.

Scénarios fixes et aléatoires : Les scénarios aléatoires sont générés automatiquement à chaque épisode en faisant varier la position des obstacles, de l'agent et de la cible, tout en respectant les contraintes de génération (absence de recouvrement, distance minimale entre les objets, etc.). Ils permettent d'évaluer la capacité de l'agent à généraliser à des configurations qu'il n'a jamais rencontrées au cours de l'entraînement. La difficulté peut être ajustée en faisant varier le nombre d'obstacles statiques et dynamiques.

Scénarios de type entrepôt (warehouse) : Les scénarios de type warehouse reproduisent un environnement structuré composé d'allées séparées par des obstacles rectangulaires représentant des rayonnages. Ils permettent d'évaluer la capacité de l'agent à planifier des trajectoires dans des espaces contraints, où les possibilités de déplacement sont limitées et où les passages sont plus étroits. Ce type d'environnement se rapproche des configurations rencontrées dans les systèmes automatisés de transport.

Scénarios de type circulaire (crossing) : Les scénarios de type crossing sont conçus pour générer de nombreuses interactions entre les agents et les obstacles mobiles. Les agents sont disposés sur la périphérie de l'environnement et doivent rejoindre une cible située de l'autre côté, ce qui conduit à des trajectoires qui se croisent au centre de la scène. Ces scénarios permettent d'évaluer la robustesse de la politique dans des situations fortement conflictuelles, caractérisées par un risque élevé de collision et nécessitant des comportements d'évitement efficaces.

TABLE 2 – Récapitulatif des scénarios d'évaluation

<i>Évaluation</i>	<i>Description</i>	W_{env}	H_{env}	N_a	N_s	N_m	N_g	T_{max}
—	—	—	—	—	—	—	—	—

3.3 Critères d'évaluation

4 Résultats

Références

- [1] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic : Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870. PMLR, 10–15 Jul 2018.
- [2] OpenAI. Soft actor-critic, 2023. Spinning Up documentation, consulté le 5 mai 2026.
- [3] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan. Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [4] J. Choi, G. Lee, and C. Lee. Reinforcement learning-based dynamic obstacle avoidance and integration of path planning. *International Journal of Intelligent Robotics and Applications*, 14(5) :663–677, 2021.
- [5] Z. Huang, Z. Ren, T. Chen, S. Cai, and C. Xu. Autonomous navigation and collision avoidance for agv in dynamic environments : An enhanced deep reinforcement learning approach with composite rewards and dynamic update mechanisms. *IET Cyber-Systems and Robotics*, 2024.
- [6] G. Chen, S. Yao, J. Ma, L. Pan, Y. Chen, P. Xu, J. Ji, and X. Chen. Distributed non-communicating multi-robot collision avoidance via map-based deep reinforcement learning. *Sensors*, 20(17) :4836, 2020.

Table des figures

1	Architecture générale de l'extracteur de caractéristiques convolutionnel	9
2	Architecture générale de l'actor network	9
3	Architecture générale des critic networks	10

Liste des tableaux

1	Récapitulatif des scénarios d'entraînement	12
2	Récapitulatif des scénarios d'évaluation	14
3	Paramètres du robot	18
4	Paramètres de la fonction de récompense	18
5	Paramètres du modèle	18

A Annexes

A.1 Paramètres du simulateur

TABLE 3 – Paramètres du robot

<i>Paramètre</i>	<i>Définition</i>	<i>Val.</i>
K	Nombre de cartes locales utilisées	1
W	Largeur du champ de vision en mètres	11
H	Hauteur du champ de vision en mètres	11
r	Rayon du robot en mètres	0.5
$v_{\min}$	Vitesse linéaire minimale en m/s	0.0
$v_{\max}$	Vitesse linéaire maximale en m/s	5.0
$\omega_{\min}$	Vitesse angulaire minimale en radian/s	$-\pi/2$
$\omega_{\max}$	Vitesse angulaire maximale en radian/s	$\pi/2$

TABLE 4 – Paramètres de la fonction de récompense

<i>Paramètre</i>	<i>Définition</i>	<i>Val.</i>
β_1	Poids de la récompense de progression	5.0
β_2	Poids de la pénalité de rotation	0.1
β_3	Poids de la pénalité de sécurité	0.5
r_{goal}	Bonus d'atteinte de la cible	400.0
$r_{\text{collision}}$	Pénalité de collision	-100.0
r_{angular}	Coefficient de pénalité angulaire	-2.0
r_{safety}	Coefficient de pénalité de sécurité	-1.0
ξ_s	Distance de sécurité	2.0
ξ_g	Seuil d'atteinte de la cible en mètres	0.5
ξ_c	Seuil de collision en mètres	0.5

TABLE 5 – Paramètres du modèle

<i>Paramètre</i>	<i>Définition</i>	<i>Val.</i>
γ	Facteur d'actualisation	0.990
α	Coefficient d'entropie	0.150
λ	Coefficient de soft update	0.005
η_{actor}	Coefficient d'apprentissage de l'actor network	3×10^{-5}
η_{critic}	Coefficient d'apprentissage des critic networks	3×10^{-4}
N_{batch}	Taille du mini-batch	64
N_{memory}	Taille maximale du replay buffer	100 000

A.2 Pseudo-code de l'algorithme SAC

Algorithm 1: Soft Actor-Critic

inputs :

θ : paramètres initiaux de la politique
 ϕ_1, ϕ_2 : paramètres initiaux des fonctions de valeurs d'action
 $\mathcal{D}$: replay buffer vide
 $\eta_{\text{critic}}, \eta_{\text{actor}}$: taux d'apprentissage

initialisation : $\phi_{1,\text{target}} \leftarrow \phi_1, \phi_{2,\text{target}} \leftarrow \phi_2$

On applique la procédure jusqu'à convergence...

for chaque *itération* **do**

for chaque *étape* dans l'environnement **do**

On observe s puis on exécute a dans l'environnement

$\mathbf{a} \sim \pi_{\theta}(\cdot \mid \mathbf{s})$

$\mathbf{s}' \sim P(\cdot \mid \mathbf{s}, \mathbf{a})$

$r \leftarrow R(\mathbf{s}, \mathbf{a})$

On stocke les transitions dans le replay buffer

$\mathcal{D} \leftarrow \mathcal{D} \cup (\mathbf{s}, \mathbf{a}, r, \mathbf{s}', d)$

Si s' est terminal, on recommence

if $d = 1$ **then** réinitialiser l'environnement

for chaque *étape* de mise à jour **do**

Échantillonner un batch de transitions aléatoirement

$B \sim \mathcal{D}$

Calculer les targets des fonctions Q

$\tilde{\mathbf{a}}' \sim \pi_{\theta}(\cdot \mid \mathbf{s}')$

$y \leftarrow r + \gamma(1 - d) \left(\min_{i \in \{1, 2\}} Q_{\phi_{i,\text{target}}}(\mathbf{s}', \tilde{\mathbf{a}}') - \alpha \log \pi(\tilde{\mathbf{a}}' \mid \mathbf{s}') \right)$

Mettre à jour Q_{ϕ_1} et Q_{ϕ_2} par descente de gradient, avec :

$\phi_i \leftarrow \phi_i - \eta_{\text{critic}} \nabla_{\phi_i} \frac{1}{|B|} \sum_{(\mathbf{s}, \mathbf{a}, r, \mathbf{s}', d) \in B} (Q_{\phi_i}(\mathbf{s}, \mathbf{a}) - y)^2$

Mettre à jour π_{θ} par montée de gradient, avec :

$\varepsilon \sim \mathcal{N}(0, I)$

$\tilde{\mathbf{a}}_{\theta}(\mathbf{s}, \varepsilon) \leftarrow \tanh(\mu_{\theta}(\mathbf{s}) + \sigma_{\theta}(\mathbf{s}) \odot \varepsilon)$

$\theta \leftarrow \theta + \eta_{\text{actor}} \nabla_{\theta} \frac{1}{|B|} \sum_{\mathbf{s} \in B} (\min_{i \in \{1, 2\}} Q_{\phi_i}(\mathbf{s}, \tilde{\mathbf{a}}_{\theta}) - \alpha \log \pi_{\theta}(\tilde{\mathbf{a}}_{\theta} \mid \mathbf{s}))$

Mettre à jour les target networks :

$\phi_{1,\text{target}} \leftarrow (1 - \lambda)\phi_{1,\text{target}} + \lambda\phi_1$

$\phi_{2,\text{target}} \leftarrow (1 - \lambda)\phi_{2,\text{target}} + \lambda\phi_2$
